Linux

What is Linux?

Linux is an open-source, Unix-like operating system kernel designed to provide a stable, secure, and efficient environment for running applications. It is widely used in servers, desktops, mobile devices, embedded systems, and cloud platforms due to its flexibility, scalability, and robust security features.

How Linux Works

1. **Kernel:** The core of Linux, responsible for managing hardware resources and enabling communication between hardware and software. It handles processes, memory, file systems, and device drivers.
2. **Shell:** The interface that allows users to interact with the kernel via commands.
3. **File System:** Organizes data hierarchically, starting from the root directory (/) and branching into subdirectories.
4. **Processes:** Managed using process IDs (PIDs), Linux handles multitasking by efficiently scheduling and managing active processes.
5. **Hardware Abstraction:** Linux abstracts hardware through device drivers, enabling the same kernel to run on different hardware architectures.
6. **Users and Permissions:** Implements multi-user access and permissions to ensure security and isolation.

Popular Linux Distributions

1. **Ubuntu**: User-friendly, widely used for desktops and servers, and suitable for beginners.
2. **CentOS/AlmaLinux/RHEL**: Enterprise-level distributions, known for stability and security in production environments.
3. **Debian**: A stable and versatile distribution, often used as a base for others like Ubuntu.
4. **Fedora**: A cutting-edge distribution with the latest features, suitable for developers.
5. **Arch Linux**: A lightweight and customizable distribution, designed for advanced users.
6. **Kali Linux**: Specializes in penetration testing and cybersecurity tasks.
7. **SUSE Linux Enterprise**: Offers enterprise-level stability and professional support.
8. **Linux Mint**: Focused on simplicity and ease of use, ideal for beginners.

Opskillsup

Linux File Structure

```
/
├── bin
├── boot
├── dev
├── etc
├── home
├── lib
├── media
├── mnt
├── opt
├── proc
├── root
├── run
├── sbin
├── srv
├── sys
├── tmp
├── usr
└── var
```

Explanation of Directories

1. **/ (Root Directory)**
    - The top-level directory of the Linux file system hierarchy.
    - Contains all other directories and files.
    - Only root (superuser) has full permissions to modify its contents.
2. **/bin (Binary)**
    - Contains essential executable binaries (commands) required for the system to boot and operate.
    - Examples: ls, cp, mv, bash.

Opskillsup

3. **/boot**
    - Contains files needed to boot the system, such as the kernel and bootloader configuration.
    - Example: vmlinuz (compressed kernel image), grub (bootloader).
4. **/dev (Device Files)**
    - Contains special device files that represent hardware devices.
    - Examples: sda (disk), tty (terminal).
5. **/etc (Configuration Files)**
    - Stores system-wide configuration files and shell scripts for initialization.
    - Examples: /etc/passwd, /etc/fstab.
6. **/home**
    - Contains user-specific directories and files.
    - Example: /home/username stores personal files for the user "username."
7. **/lib (Libraries)**
    - Contains shared libraries required by system programs in /bin and /sbin.
    - Example: libc.so.6 (standard C library).
8. **/media**
    - Temporary mount points for removable media, such as USB drives or CDs.
9. **/mnt**
    - Temporary mount point for file systems (often used by system administrators).
10. **/opt (Optional Software)**
    - Stores additional software packages installed on the system.
11. **/proc (Process Information)**
    - Virtual file system providing information about running processes.
    - Example: /proc/cpuinfo (CPU details), /proc/meminfo (memory details).
12. **/root**
    - Home directory for the root (superuser) account.

opskillsup

13. **/run**

- Temporary runtime data for the system and applications (e.g., PID files, sockets).

14. **/sbin (System Binaries)**

- Contains system administration binaries for tasks like system recovery.
- Examples: ifconfig, fdisk.

15. **/srv (Service Data)**

- Stores data for services such as web servers or FTP servers.

16. **/sys (System Information)**

- Virtual file system containing information about the hardware and kernel.
- Similar to /proc, but focuses on devices and drivers.

17. **/tmp (Temporary Files)**

- Temporary files created by applications and the system.
- Cleared on system reboot.

18. **/usr (User Binaries and Data)**

- Contains user-installed programs, libraries, and documentation.
- Subdirectories:
    - /usr/bin (user binaries),
    - /usr/lib (user libraries),
    - /usr/share (shared data).

19. **/var (Variable Data)**

- Contains data that changes frequently during system operation.
- Examples: Logs (/var/log), caches (/var/cache), spool files (/var/spool).

Opskillsup

Commands

ls Command

The ls command is used to list files and directories in the current directory or a specified path.

Commonly Used Flags for ls

Flag	Description	Example
`-l`	Displays detailed (long) listing, including permissions, owner, group, size, and modification time.	`ls -l`
`-a`	Displays all files, including hidden files (those starting with `.`).	`ls -a`
`-h`	Displays file sizes in a human-readable format (e.g., KB, MB) when used with `-l`.	`ls -lh`
`-R`	Recursively lists subdirectories and their contents.	`ls -R`
`-t`	Sorts files by modification time, with the newest first.	`ls -t`
`-r`	Reverses the order of sorting (e.g., reverse alphabetical).	`ls -r`
`-d`	Lists only directories, not their contents.	`ls -d */`
`-S`	Sorts files by size, with the largest first.	`ls -S`
`-1`	Displays one file per line.	`ls -1`
`-F`	Appends symbols to indicate file types (e.g., `/` for directories, `*` for executables).	`ls -F`
`--color`	Adds color coding to file types for be visualization.	`ls --color`

Detailed Examples

Basic Usage:

1. ls - List files and directories in the current directory.

2. ls -l - Show detailed information about files and directories.

Output Example:

```
-rw-r--r--  1 user group   1024 Dec 27 10:00 file.txt
drwxr-xr-x  2 user group   4096 Dec 26 09:30 my_folder
```

3. ls -a - Include hidden files (starting with .).

4. ls -lh - Combine -l and -h to see file sizes in KB, MB, or GB.

Output Example:

```
-rw-r--r--  1 user group   1.0K Dec 27 10:00 file.txt
drwxr-xr-x  2 user group   4.0K Dec 26 09:30 my_folder
```

5. ls -R - Show all files and directories, including subdirectories.

6. ls -lt - Sort files by their modification time (newest first).

7. ls -lrr - Reverse the sort order

8. ls -d */ - Show only directories in the current location.

9. ls -lS - Display files sorted by size, largest first.

10. ls -F – visual indicator: Show symbols indicating file types (/ for directories, * for executables).

Output Example:

```
file.txt
script.sh*
my_folder/
```

11. ls -alh - This displays all files (including hidden) in a detailed, human-readable format.

Advanced Flags

Flag	Description	Example
`--help`	Displays help for the `ls` command.	`ls --help`
`--full-time`	Shows full timestamp for files.	`ls --full-time`
`--group-directories-first`	Lists directories first, then files.	`ls --group-directories-first`
`--hide=<pattern>`	Hides files matching a pattern (e.g., `*.log`).	`ls --hide=*.log`
`--version`	Shows version information for the `ls` command.	`ls --version`

- **Pwd** – present working directory
- **Mkdir** – creating a new directory
- **Touch** – used to create new empty file
- **Cd** – change directory
- **Cd ..** –to come out of present directory
- **Cd ../..** – if you want to come out of 2 or more directories.
- **Cd - :** acts as a recall between last 2 commands
- **Cd /temp1/temp2/** - changes to one directory to last directory
- **Cat** – used to display the content of a file on console
- **Cat -n filename** – disaplay the content of a file with line numers on console

- **Vi – used to edit a file on vi editor**

  - Esc I – insert mode in vi editor.
  - Esc :wq! – save and quit the file. W-write,q-quit,!-forcefully.
  - Esc q! – quit without saving the content.
  - Esc :set number/ esc :setnu – which sets the number for a file.
  - Esc :setnonu – used to remove the line numbers for a file.
  - Replace a string ♣ Esc :%s/string1/string2/g
  - G is not mandatory, **g** means globally.
  - Esc :%s/linux/docs/g **S** is substitute
  - Esc :1 s/linux/docs/g – changes in 1st line only
  - Esc :1,5 s/linux/docs/g –changes 1 to 5th line.
  - Esc :7,$ s/linux/docs/g – changes 7th to end of line.
  - Esc :1 s/linux/docs/1 – changes 1st line 1st occurrence only.
  - Delete the whole line Esc:dd

Du(disk usage) – use to check the size of a file

Syntax:

du -sh filename

du -sh * - for all the files

df -h – to check the current disk size

free -h – to check the system memory

| (Pipe): The pipe (|) command in Linux is used to take the output of one command and use it as the input for another command. It allows you to combine multiple commands to perform more complex tasks efficiently.

Opskillsup

- **mv** <old_filename.txt> <new_filename.txt> - To rename a file in the same directory
- **mv** /path/to/source/file.txt /path/to/destination/ - To move a file from one directory to another:

Common Flags for `mv`

Flag	Description	Example
`-i`	Prompts before overwriting an existing file.	`mv -i file1.txt file2.txt`
`-f`	Forces the operation without prompting, even if the destination exists.	`mv -f file1.txt file2.txt`
`-n`	Prevents overwriting of existing files.	`mv -n file1.txt file2.txt`
`-u`	Moves only when the source file is newer than the destination file.	`mv -u file1.txt file2.txt`
`-v`	Displays verbose output, showing the files being moved or renamed.	`mv -v file1.txt file2.txt`
`--backup`	Creates a backup of the destination file if it exists.	`mv --backup file1.txt file2.txt`
`--suffix=<suffix>`	Adds a custom suffix to backups (used with `--backup`).	`mv --backup --suffix=.old file1.txt file2.txt`
`--strip-trailing-slashes`	Removes trailing slashes from the source directory path.	`mv --strip-trailing-slashes dir1/ dir2`

> (Redirect): Redirects output to a file, **overwriting** its existing content. Creates the file if it doesn't exist.

echo "Hello" > file.txt # Replaces content of file.txt with "Hello"

>> (Append): Redirects output to a file, **appending** to its existing content without overwriting. Creates the file if it doesn't exist.

echo "World" >> file.txt # Adds "World" to the end of file.txt

---

cp source.txt /home/user/destination/ - Copy a file from one location to another.

cp source.txt newfile.txt - Copy a file to a new location with a different name.

cp -r /home/user/dir1 /home/user/dir2 - Copy a directory along with its contents

cp -i source.txt destination.txt - Prompt before overwriting files.

cp -p source.txt /home/user/destination/ - Copy files while preserving attributes like timestamps and permissions.

Grep

Grep Command: used to search a word or pattern in a file

grep "hello" file.txt - Search for a specific string within a file.

grep -r "hello" /home/user/ - Search for a pattern within all files in a directory and its subdirectories.

grep -i "hello" file.txt - Perform a case-insensitive search.

grep -n "hello" file.txt - Show the line numbers where the pattern matches.

grep -v "hello" file.txt - Show lines that do not contain the specified pattern.

grep -c "hello" file.txt - Count how many times a pattern appears in the file.

grep -w "hello" file.txt - Search for a complete word (not part of another word).

grep -e "hello" -e "world" file.txt - Search for one or more patterns using -e.

grep "hello" file1.txt file2.txt - Search in multiple files at once.

grep "hello" *.txt - Use grep with wildcard or regular expression to search in files matching a pattern.

Common `grep` Flags

- `-r` : Recursive search in directories and subdirectories.
- `-i` : Case-insensitive search.
- `-n` : Show line numbers with matching lines.
- `-v` : Invert match (show lines that don't match).
- `-c` : Count the number of matches.
- `-w` : Match only whole words.
- `-e` : Specify multiple patterns to search for.
- `-l` : Show only filenames of matching files.
- `-H` : Show filename along with matching lines.
- `-o` : Show only the matched parts of the line.

Head and Tail command

head file.txt - By default, head shows the first 10 lines of a file.

head -5 file.txt - specify the number of lines to display.

head file1.txt file2.txt - head with multiple files to show the first few lines of each.

tail file.txt - By default, tail shows the last 10 lines of a file.

tail -5 file.txt - Use the -n flag to specify the number of lines to display from the end.

tail -f /var/log/syslog - Use the -f flag to continuously monitor a file for new content (useful for logs).

tail file1.txt file2.txt - Use tail with multiple files to show the last few lines of each.

head -10 file.txt | tail -6 - Show lines 5-10 from file.txt

head -99 filename | tail -1 - Shows 99th line of a file

tail -3 filename | head -2 – Shows last 2nd and 3rd line of a file

head -8 filename | tail -1 | wc -c – count no of character in 8th line

---

Sed Command

Sed - used to replace a string in file

sed 's/foo/bar/' file.txt – Replace the first occurrence of a word in each line

sed 's/foo/bar/g' file.txt - Replace all occurrences of a word in each line

sed '3d' file.txt - Delete a specific line by number

sed '/pattern/d' file.txt - Delete lines matching a pattern

sed -n '5p' file.txt - Print the 5th line of a file

sed '1i Inserted Text' file.txt - Insert text before a specific line (e.g., before the 1st line)

sed '$a Appended Text' file.txt - Append text after a specific line (e.g., after the last line)

sed '2c This is the new line text' file.txt - Replace the entire 2nd line with new text

sed -i 's/foo/bar/g' file.txt - Replace text directly in the original file

sed '5,10d' file.txt - Delete lines from 5 to 10

opskillsup

Flag	Description
`-e`	Allows multiple commands in one `sed` invocation.
`-n`	Suppresses automatic printing of lines; use with `p` to print specific lines.
`-i`	Edits the file in place (modifies the original file).
`g`	Replaces all occurrences of a pattern in a line (global).
`p`	Prints the modified or matched lines (useful with `-n`).

Find Command

The **find** command is used to search for files and directories in a directory hierarchy based on various criteria like name, type, size, permissions, etc.

find . -name "example.txt" - Search for files named example.txt

find . -iname "example.txt" - Search for files named example.txt (case-insensitive)

find . -type d -name "backup" - Search for directories named backup

find . -type f -size +100M - Find files larger than 100MB:

find . -type f -size -1k - Find files smaller than 1KB

find . -type f -mtime -7 - Find files modified in the last 7 days

find . -type f -perm 644 - Find files with 644 permissions

find . -type f -name "*.log" | xrags rm –rf - Delete all .log files

find . -type f -name "*.tmp" –delete - Delete all .tmp files

find . -maxdepth 1 -name "*.txt" - Search only in the current directory (no subdirectories)

find /home -user john - Find files owned by the user john

Opskillsup

Common Flags for `find`

Flag	Description
`-name`	Search for files/directories by name (case-sensitive).
`-iname`	Search for files/directories by name (case-insensitive).
`-type`	Specifies the type of file (e.g., `f` for regular file, `d` for directory).
`-size`	Finds files based on size (e.g., `+1M` for greater than 1MB).
`-perm`	Finds files with specific permissions (e.g., `-perm 644`).
`-user`	Finds files owned by a specific user.
`-group`	Finds files owned by a specific group.
`-mtime`	Finds files modified within a specific number of days (e.g., `-mtime -7`).
`-atime`	Finds files accessed within a specific number of days.
`-ctime`	Finds files changed within a specific number of days.
`-exec`	Executes a command on the found files.
`-delete`	Deletes files/directories that match the criteria.
`-maxdepth`	Limits the search to a specific number of directory levels.
`-mindepth`	Skips searching in the first `n` directory levels.

AWK Command

1. Print Specific Columns

awk '{print $1}' file.txt - Print the first column from a file:

awk '{print $1, $3}' file.txt - Print the first and third columns from a file:

2. Filter Lines Based on Column Values

awk '$1 == "error" {print $0}' file.txt - Print lines where the first column equals 'error':

awk '$2 > 100 {print $0}' file.txt - Print lines where the second column is greater than 100:

3. Perform Simple Calculations

awk '{sum += $2} END {print sum}' file.txt - Sum the values in the second column:

awk '{sum += $2; count++} END {print sum/count}' file.txt - Average of the second column:

4. Print Lines with Line Numbers

awk '{print NR, $0}' file.txt - Print the line number followed by the line content:

5. Modify Field Separator

awk -F, '{print $1, $2}' file.csv - If the file is comma-separated, use `-F` to specify the field separator:

6. Use awk with Text Filtering

awk '/error/ {print $0}' file.txt - Print lines that contain the word 'error':

7. Set Output Field Separator

awk 'BEGIN {OFS=","} {print $1, $2}' file.txt - Change the output field separator to a comma:

8. Pass Variables to awk

Use an external variable to filter data:

threshold=50

awk -v threshold="$threshold" '$2 > threshold {print $0}' file.txt

9. Process Multiple Files

awk '{print $1}' file1.txt; awk '{print $1}' file2.txt - Process two files and print columns from each:

10. Print Total Number of Lines

awk 'END {print NR}' file.txt - Count the total number of lines in a file:

Chmod (Change File Permissions)

The `chmod` command is used to change the permissions of a file or directory.

Syntax:

chmod [options] mode file

Common Permissions:

r: Read permission (**4**), **w:** Write permission (**2**), **x:** Execute permission (**1**)

Common Flags:

-R: Recursively change permissions for directories and their contents.

+: Adds the specified permission, -: Removes the specified permission, =: Sets the exact permissions.

Examples:

1. Grant Read, Write, and Execute Permissions to User:

chmod u+rwx file.txt

2. Remove Execute Permission for Others:

chmod o-x file.txt

3. Recursively Change Permissions (Grant Read, Write, and Execute to User):

chmod -R u+rwx directory/

4. Set Permissions Using Numeric Mode (755):

chmod 755 file.txt

Chown (Change File Owner and Group)

The `chown` command is used to change the owner and/or group of a file or directory.

Syntax:

chown [options] user:group file

Common Flags:

-R: Recursively change the ownership for directories and their contents.

-v: Verbosely display the changes made.

Examples:

1. Change Owner of a File:

chown user file.txt

2. Change Owner and Group of a File:

chown user:group file.txt

3. Recursively Change Ownership of a Directory:

chown -R user:group directory/

umask (Set File Creation Mask)

The `umask` command is used to set default file creation permissions for newly created files and directories.

Syntax:

umask [mode]

Common Flags:

-S: Display the current umask value in symbolic format.

Examples:

1. Display Current umask Value:

umask

Opskillsup

2. Set umask to 0777 (No Permissions for New Files):

umask 0777

3. Set umask to 0022 (Read and Execute Permissions for Others):

umask 0022

4. Display umask in Symbolic Format:

umask -S

~/.BASHRC

- **Purpose**: The ~/.bashrc file is executed for **interactive non-login shells**. This means it is executed every time you open a new terminal or start a new shell.

  Common Use Cases:

  - Setting **aliases** (e.g., alias ll='ls -la').
  - Customizing the **command prompt** (e.g., PS1='[\u@\h \W]\$ ').
  - Enabling **shell features** like tab completion and history settings.
  - Setting environment variables that are needed only for the current shell session.

~/.BASH_PROFILE

- **Purpose**: The ~/.bash_profile file is executed for **login shells**, which typically occurs when you log in to a system via SSH, or if you start a terminal with login shell settings.

  Common Use Cases:

  - Setting **global environment variables** (e.g., export PATH=$PATH:/usr/local/bin).
  - Configuring **system-wide paths** and **directories**.
  - Running commands or programs that should be executed **once** when logging into the system.
  - Sourcing ~/.bashrc to ensure the same configurations are applied to login shells as well.

opskillsup

Key Differences

Aspect	`~/.bashrc`	`~/.bash_profile`
Type of Shell	Non-login interactive shells	Login shells (e.g., SSH or terminal login)
Execution Time	Every time a new shell is opened (non-login shell)	Only once when the user logs into the system
Common Use	Shell options, aliases, prompt, session settings	Environment variables, paths, login configurations
Sourcing	Not sourced by other files	Can source `~/.bashrc` for uniform settings

SSH connection between two servers using a **PEM key**

To establish an SSH connection between two servers using a PEM key (Private Key), follow these steps:

1. Prerequisites

• Ensure you have the PEM key file (e.g., key.pem) for the server you are connecting to.

2. Set Correct Permissions for PEM Key

Ensure the private PEM key has the correct permissions. The private key file should be readable only by the user:

chmod 400 key.pem

This will prevent other users from accessing the private key.

3. Establish SSH Connection

To SSH into the remote server using the PEM key, use the following syntax:

ssh -i /path/to/key.pem username@remote-server-ip

• **-i:** Specifies the path to your PEM private key file.

• **username:** The username on the remote server (e.g., ec2-user, ubuntu, root).

• **remote-server-ip:** The public IP address or hostname of the server you're connecting to.

Example:

ssh -i /path/to/key.pem ec2-user@54.123.45.67

4. First Time Connection Warning

The first time you connect, you'll get a message asking if you want to add the server to your known hosts:

The authenticity of host 'remote-server-ip' can't be established.

Opskillsup

Are you sure you want to continue connecting (yes/no)?

Type yes to continue.

5. Verifying the Connection

Once connected, you should see the shell prompt for the remote server. You can now run commands as you would on the server directly.

Troubleshooting Tips:

• Permission Denied Error: If you see Permission denied (publickey) error, check that the correct public key is in the ~/.ssh/authorized_keys file on the remote server.

• Ensure SSH Daemon is Running: Make sure the SSH service is running on the remote server (check with sudo service ssh status or systemctl status ssh).

How to establish password less between two servers

If you don't have the password for the remote server and still need to set up a passwordless SSH connection, you can manually copy the public key to the server. Here's how you can do it without needing a password:

Steps to Set Up Passwordless SSH Without Password

1. Generate the SSH Key Pair

On your local machine, generate an SSH key pair:

ssh-keygen -t rsa -b 4096

This will create two files:

Private Key: ~/.ssh/id_rsa

Public Key: ~/.ssh/id_rsa.pub

Leave the passphrase empty when prompted for one.

2. Manually Copy the Public Key to the Remote Server

Once you have access to the server (via a management console or other means), do the following:

Display the Public Key on your local machine:

cat ~/.ssh/id_rsa.pub

Access the Remote Server (via console or other means) and log in.

On the remote server, create the .ssh directory in your home folder (if it doesn't already exist):

mkdir -p ~/.ssh

chmod 700 ~/.ssh

Append the public key to the authorized_keys file:

echo "your-public-key-content" >> ~/.ssh/authorized_keys

chmod 600 ~/.ssh/authorized_keys

Replace your-public-key-content with the actual content of your id_rsa.pub (the public key you generated earlier).

3. Test the Connection

Once the public key is copied, you can now test the passwordless SSH connection:

ssh username@remote-server-ip

You should be able to log in without entering a password.

---

sudo and su Commands in Linux

sudo Command

- **Purpose:** Run a single command as another user (typically as the superuser/root).
- **Example:**

  sudo apt update

  This updates the package list with root privileges.

Explanation:

- sudo stands for "superuser do."
- It allows a permitted user to execute a command as the superuser or another user.

su Command

- **Purpose**: Switch to another user account or the superuser account.
- **Example**:

  su

This switches to the root user account. You'll need to provide the root password.

- **Switch to a specific user**:

  su – username

  This logs in as the specified user, initiating their shell environment.

Difference Between sudo and su

- sudo is used to execute individual commands with elevated privileges.
- su is used to switch to another user account or root entirely.

How to Add a User in Linux

1. Using adduser Command (Simpler Method)

- **Example**:

  sudo adduser newuser

  Follow the prompts to set a password and optional user details.

2. Using useradd Command (Advanced Method)

- **Example**:

  sudo adduser newuser

- **-m:** Creates a home directory for the user.
- **-s** /bin/bash: Sets the default shell for the user.
- **passwd:** Sets a password for the new user

Granting sudo Privileges to the New User

1. Add User to sudo Group

sudo usermod -aG sudo newuser

- **-aG sudo: Appends the user to the sudo group.**

ps -ef Command in Linux

The ps -ef command is used to display all the processes running on a Linux system with detailed information in a full-format listing.

Common Flags for `ps`

1. `-e`: Displays all processes running on the system, not limited to the current terminal.
2. `-f`: Provides a full-format listing that includes detailed information about each process, such as:
   - UID: User ID of the process owner
   - PID: Process ID
   - PPID: Parent Process ID
   - C: CPU utilization
   - STIME: Process start time
   - TTY: Terminal associated with the process
   - TIME: CPU time used by the process
   - CMD: Command that started the pro

ps -ef | grep username - Find Processes by User

ps -ef | grep process_name - Find Specific Processes by Name

kill -9 PID – Kill process forcefully

How to start stop restart process gracefully

sudo systemctl start service_name - Start a Service

sudo systemctl stop service_name - Stop a Service

sudo systemctl restart service_name – restart a service

sudo systemctl reload service_name - Reload a Service Configuration

kill -15 PID - Stop/Terminate Gracefully

sort and uniq

`sort` Command in Linux

The `sort` command is used to sort lines of text files or input in ascending or descending order.

Basic Syntax:

sort [options] filename

Common Use Cases and Examples:

Sort a File Alphabetically:

Command:

sort file.txt

Example Input (`file.txt`):

```
apple
banana
cherry
apple
```

Output:

```
apple
apple
banana
cherry
```

2. Sort in Reverse Order:

Command:

sort -r file.txt

Output

```
cherry
banana
apple
apple
```

3. Sort by Numeric Order:

Command:

sort -n numbers.txt

Example Input (`numbers.txt`):

```
10
2
30
25
```

Output:

```
2
10
25
30
```

sort -f file.txt - Sort with Case-Insensitive Comparison

The `**uniq**` command is used to filter out or report duplicate lines in a sorted file or input. It works best when used with `sort`

Basic Syntax:

uniq [options] filename

Common Use Cases and Examples:

1. Remove Duplicate Lines:

Command:

sort file.txt | uniq

Example Input (`file.txt`):

```
apple
apple
banana
cherry
```

Output:

```
apple
banana
cherry
```

2. Count Duplicates:

Command:

sort file.txt | uniq -c

Output:

```
2 apple
1 banana
2 cherry
```

3. Show Only Duplicate Lines:

Command:

sort file.txt | uniq -d

Output:

```
apple
cherry
```

4. Show Only Unique Lines:

Command:

sort file.txt | uniq -u

opskillsup

Output:

```
banana
```

Combining `sort` and `uniq`

1. Sort and Remove Duplicates:

Command:

sort file.txt | uniq

2. Count Unique Lines in a File:

Command:

sort file.txt | uniq | wc -l

Additional Options

`sort` **Flags:**

- `-n` : Sort numerically.
- `-r` : Reverse order.
- `-f` : Case-insensitive sort.
- `-k` : Specify a key for sorting (e.g., by column).

`uniq` **Flags:**

- `-c` : Count duplicates.
- `-d` : Display only duplicate lines.
- `-u` : Display only unique lines.

Which command - Locate the full path of an executable file in the system.

uname -a - Show complete system details

uname -m - Display only the machine architecture weather it is 64 or 32 bit

tar -cvf archive_name.tar /path/to/directory_or_file - To tar a file (create a tarball)

- **-c** creates a new archive.
- **-v** shows the progress (optional, for verbose).
- **-f** specifies the file name for the archive.

tar -xvf archive_name.tar - To untar a file (extract the tarball)

- **-x** extracts files from the archive.
- **-v** shows the progress (optional).
- **-f** specifies the tarball name.

tar -xzvf archive_name.tar.gz - If the archive is compressed with gzip (e.g., .tar.gz or .tgz)

tar -xjvf archive_name.tar.bz2 - For bzip2 compression (.tar.bz2)

Top - This command will show a dynamic list of processes currently running on your system, updating every few seconds.

Useful options:

- `top -u <user>`: Display processes for a specific user.
- `top -p <pid>`: Display information for a specific process ID.
- `top -d <seconds>`: Change the refresh interval (default is 3 seconds).
- `top -n <count>`: Set the number of updates before exiting.

Htop - This command will open an interactive display of system processes. You can scroll through the list, search for processes, and kill them directly from the interface.

Useful options:

- `htop -u <user>`: Display processes for a specific user.
- `htop -p <pid>`: Display processes for a specific process ID.
- `htop -s <sort_column>`: Sort the display by a specific column (e.g., CPU, memory usage).

<u>curl Command</u>

Example 1: Fetching a Web Page with curl

Command:

curl https://example.com

Explanation: Fetches the content of the URL "https://example.com" and displays it in the terminal.

Example 2: Downloading a File with curl

Command:

curl -O https://example.com/file.zip

Explanation: Downloads the file "file.zip" from "https://example.com" and saves it with the same name.

Example 3: Sending a POST Request with curl

Command:

curl -X POST https://example.com/api/data -d '{"name":"John", "age":30}' -H "Content-Type: application/json"

Explanation: Sends a POST request to the specified API with a JSON body and header specifying content type as JSON.

Example 4: Saving Output to a File with curl

Command:

curl https://example.com -o webpage.html

Explanation: Downloads content from "https://example.com" and saves it as "webpage.html".

wget Command

Example 1: Downloading a File with wget

Command:

wget https://example.com/file.zip

Explanation: Downloads the file "file.zip" from "https://example.com".

Example 2: Downloading a File and Saving It with a Specific Name using wget

Command:

wget -O custom_name.zip https://example.com/file.zip

Explanation: Downloads the file and saves it as "custom_name.zip" instead of the original filename.

Example 3: Downloading an Entire Website using wget

Command:

wget -r -np -k https://example.com/

Explanation: Recursively downloads files from the website, excluding files from parent directories, and converts links for offline viewing.

Example 4: Downloading Files in the Background using wget

Command:

wget -b https://example.com/largefile.zip

Explanation: Downloads the file in the background, freeing up the terminal for other tasks.

Opskillsup

Example 5: Limiting Download Speed with wget

Command:

wget --limit-rate=200k https://example.com/largefile.zip

Explanation: Limits the download speed to 200 KB/s.

Key Differences Between curl and wget

1. curl is best for interacting with APIs and sending HTTP requests (GET, POST, etc.), while wget is used for downloading files or entire websites.

2. curl displays content by default, while wget saves the content to a file.

3. wget supports recursive downloading of websites, while curl does not support recursion natively.

4. wget can download files in the background using the -b option, whereas curl does not have native support for background downloads.

Networking commands

1. ifconfig

Example 1: Display all network interfaces

Command:

ifconfig

Explanation: Displays information about all active network interfaces including IP addresses, MAC addresses, etc.

Example 2: Display information for a specific interface

Command:

ifconfig eth0

Explanation: Displays information about the network interface "eth0".

2. ip

Example 1: Show IP addresses

Command:

ip addr show

Explanation: Displays IP addresses assigned to all network interfaces.

Example 2: Configure an IP address

Command:

Opskillsup

ip addr add 192.168.1.100/24 dev eth0

Explanation: Assigns the IP address "192.168.1.100" with subnet "/24" to interface "eth0".

3. ping

Example 1: Ping a website

Command:

ping google.com

Explanation: Sends ICMP Echo Request to "google.com" and checks its availability.

4. traceroute

Example 1: Trace the route to a website

Command:

traceroute google.com

Explanation: Shows the route packets take to reach "google.com" along with each hop time.

5. netstat

Example 1: Display network connections

Command:

netstat -tuln

Explanation: Displays all active network connections and listening ports (TCP/UDP).

- **-t:** Displays TCP connections.
- -**u:** Displays UDP connections.
- -**l:** Shows listening ports.
- **-n:** Shows numerical addresses instead of resolving them to names.

6. ss

Example 1: Display listening sockets

Command:

ss -tuln

Explanation: Similar to "netstat", shows active listening sockets and their information.

7. nslookup

Example 1: Lookup DNS for a domain

Command:

nslookup google.com

opskillsup

Explanation: Resolves "google.com" to its corresponding IP address.

8. dig

Example 1: Query DNS for a domain

Command:

dig google.com

Explanation: Fetches detailed DNS information, including A records, for "google.com".

9. route

Example 1: Display routing table

Command:

route -n

Explanation: Displays the IP routing table with numerical addresses.

10. hostname

Example 1: Display the system's hostname

Command:

hostname

Explanation: Displays the current hostname of the system.

11. iptables

Example 1: List current firewall rules

Command:

sudo iptables -L

Explanation: Displays the current firewall rules configured with iptables.

Explanation: Downloads the file "file.zip" from "https://example.com".

12. tcpdump

Example 1: Capture packets on eth0

Command:

sudo tcpdump -i eth0

Explanation: Captures and displays network packets on the "eth0" interface.

Opskillsup

13. nmap

Example 1: Scan for open ports on a target

Command:

nmap example.com

Explanation: Scans the host "example.com" for open ports and services running.

14. mtr

Example 1: Trace and monitor route to a domain

Command:

mtr google.com

Explanation: Continuously monitors the route to "google.com" and shows packet loss and latency on each hop.

Soft link and hard link

In Linux, both hard links and soft (symbolic) links are ways to create references to files,

Difference Between Soft and hard links

Feature	Hard Link	Soft Link (Symbolic Link)
Target	Points directly to the inode data	Points to the file path
Inode Number	Shares the same inode as the original file	Has its own inode, separate from the target file
Deletion of Original	Data remains accessible via the hard link	Link becomes broken if the target file is deleted
Cross Filesystems	Cannot span across different filesystems	Can point to files on different filesystems
File Types	Only files (not directories)	Can point to both files and directories
Flexibility	Less flexible	More flexible (supports symbolic paths and directories)

Syntax:

ln file1.txt link_to_file1.txt – Creates hard link to a file

ln -s /home/user/file1.txt link_to_file1.txt – Creates soft link to a file

ls -l link_to_file1.txt - The symbolic link will show the path it points to

Troubleshoot the logs of an EC2 machine, follow these general steps:

Check System Logs

- View system logs: System logs can provide general information about hardware, kernel issues, and system startup. Look into these logs:
    - /var/log/messages: General system logs.
    - /var/log/syslog (on Ubuntu/Debian): System logs.
    - /var/log/dmesg: Kernel ring buffer logs (hardware and driver-related issues).
    - /var/log/journal/ (if using systemd): View logs using journalctl for more detailed logs.

Opskillsup